

Automatic Distributed Parallelization and Communication Optimization for Stencils on Structured Grids

Anna Mittermair, Martin Schulz, Julien Remy, Hugo Brunie, Martin Schreiber

▶ **To cite this version:**

Anna Mittermair, Martin Schulz, Julien Remy, Hugo Brunie, Martin Schreiber. Automatic Distributed Parallelization and Communication Optimization for Stencils on Structured Grids. 2025. ⟨hal-05429086⟩

HAL Id: hal-05429086

https://hal.science/hal-05429086v1

Preprint submitted on 22 Dec 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Automatic Distributed Parallelization and Communication Optimization for Stencils on Structured Grids

Anna Mittermair, Martin Schulz
Technical University of Munich
Garching, Germany
anna.mittermair@tum.de
martin.w.j.schulz@tum.de

Julien Remy, Hugo Brunie, Martin Schreiber
Univ. Grenoble Alpes, Inria, CNRS, LJK, Grenoble INP
Grenoble, France
julien.remy@univ-grenoble-alpes.fr
hugo.brunie@inria.fr
martin.schreiber@univ-grenoble-alpes.fr

***Abstract*—Stencil computations on structured grids are widely used in scientific and engineering simulations. Efficiently parallelizing these computations across distributed-memory systems requires careful management of data dependencies and communication.**

We present a general approach that automatically identifies stencil operations and then generates optimized communication code, with specific adaptations for ocean modeling. Our domain-specific framework lifts sequential Fortran stencil kernels into a data-flow graph (DFG) representation. Based on this DFG, the system infers required halo exchanges, inserts the corresponding communication operations, and applies a sequence of graph transformations to reduce communication overhead through communication avoidance and communication-computation overlap. These transformations include extending halo regions, eliminating unnecessary exchanges by analyzing dependencies within and across timesteps, and splitting communication into asynchronous phases to enable overlap with computation.

Evaluation on a 2D barotropic solver demonstrates that our approach automatically parallelizes stencil computations. Additionally, the optimizations effectively reduces communication overhead, with the automatically generated code scaling significantly more efficient up to 1,024 nodes.

I. Introduction

Modern supercomputers are increasingly limited by communication latency rather than compute performance, as processor speed improves faster than network bandwidth and latency [1], [2]. Consequently, more and more scientific applications, which rely on distributed parallelization, become communication-bound. Ocean models are a prominent example of this class of applications. They are widely used in climate, weather, and coastal simulations, relying on stencil-based solvers on structured grids. The performance of these solvers is dominated by frequent halo exchanges between neighboring subdomains, which incentivizes optimizing for it by minimizing the number of halo exchanges and using asynchronous communication.

However, optimizing stencil codes for such purposes is challenging because it depends on numerous factors, including the target hardware and network, the communication library, precompiler directives, model parameters, and the code's precise data dependencies. Manually tuning communication under these constraints is challenging and prone to error, often relying on trial-and-error development that can only cover a subset of possible configurations of production ocean models. This challenge is compounded by the structure of production ocean models, which are often large and complex Fortran codebases developed and maintained by domain scientists over extended time frames.

Existing approaches to automating communication for distributed stencil computations, such as domain-specific languages (DSLs) and Partitioned Global Address Space (PGAS) languages, can reduce some of this burden. However, they often require substantial code refactoring or complete rewrites, and key aspects of communication, scheduling, and halo management frequently remain the programmer's responsibility. As a result, production ocean models and other large-scale stencil codes often struggle to determine the optimal point at which to communicate within one time step.

At the same time, existing Fortran stencil kernels already encode the information needed to automate distributed execution. Their regular loop structures and array access patterns explicitly define data dependencies and neighborhood relationships. This gives the incentive to investigate the possibility of generating communication directly from the original code and domain-specific knowledge, without introducing a separate DSL or requiring kernel rewrites. In this work, we present an automatic, communication-optimizing distributed parallelization pipeline for stencil codes on structured grids, applied directly to existing sequential Fortran kernels.

Our contributions in this paper are as follows:

- We lift sequential Fortran stencil kernels into a data-flow oriented intermediate representation,
- we perform automatic distributed parallelization and communication optimization through DFG transformations,
- we generate optimized distributed-memory Fortran code without manual kernel rewriting for one time step, and
- based on our formalism, we provide a verification mechanism for correct communication patterns.

We evaluate our approach using a research code based on the extracted barotropic timestep of the CROCO ocean model, a finite-difference ocean model using structured stencil kernels written in Fortran. This work takes up the challenge of two-dimensional stencil kernels, known to scale extremely poorly, making them an ideal target for demonstrating our approach. Such two-dimensional solvers, for example, are the solvers of barotropic modes in operational ocean models, as discussed in the next section, and are time-stepped $\mathcal{O}(50)$ times for each 3D time step.

The results show that the generated code scales to $1,024$ nodes with over $350\times$ speedup.

II. Background

This chapter introduces key concepts and terminology for both stencil codes in general and ocean models, specifically.

A. Dynamical Core of an Ocean Model

The dynamical core of an ocean model solves the discretized primitive equations of fluid dynamics, describing the evolution of key *prognostic variables*, such as velocity, free-surface height, temperature, and salinity. These PDEs are integrated forward in time from given initial conditions.

Ocean models must resolve processes on very different time scales. Fast phenomena such as barotropic gravity waves require smaller timesteps than slower three-dimensional processes. To handle this efficiently, models like CROCO [3] and NEMO [4] use a time-splitting approach [5], integrating the fast barotropic through multiple substeps solving two-dimensional problems within each larger baroclinic time step.

B. Formal Background

The following subsections introduce formal concepts relevant for communication in distributed stencil codes.

1) Spatial Discretization: Ocean models on structured grids discretize the three-dimensional spatial domain into grid cells. In the barotropic step, this grid is effectively two-dimensional with only one vertical layer. For numerical reasons, when using finite differences, grids are usually C-staggered [6]. Consequently, different variables (e.g., velocity components and free-surface height) are located at different positions within a cell. Each variable, therefore, has its own iteration range, which is further modified by boundary conditions at the grid edges. An example of iteration ranges with grid staggering and boundary conditions is shown in Figure 1.

2) Stencil Computations: A stencil computation updates each grid point using a fixed combination of neighbor values of other variables. Neighborhoods are described by *stencils*, where each stencil $S = s^M$ is a set of M offset vectors with components $s = (s_1, \ldots, s_D), s_i \in \mathbb{Z}$ describing offsets in D dimensions.

3) Parallelization and Halos: Due to their spatial locality, stencil codes can be parallelized using domain decomposition, where the global grid for a variable x is partitioned into *local domains* L_i^x, one per process P_i. For a stencil computation in process P_i to update a value of x on the edge of its local domain L_i^x, a grid element from another process P_j's local domain L_j^x may be required. To make this data accessible locally, P_i caches a copy of x in its *halo region* $H_{i,j}^x$. A *halo exchange* updates $H_{i,j}^x$ (and $H_{j,i}^x$ on P_j) by communicating boundary data between neighboring processes after their values have changed. Broader halos with multiple *halo layers* can accommodate wider stencils. Visualizations of domain decomposition and halo exchanges are shown in Figure 1. Concerning ocean models, they are primarily decomposed along the two horizontal spatial dimensions.

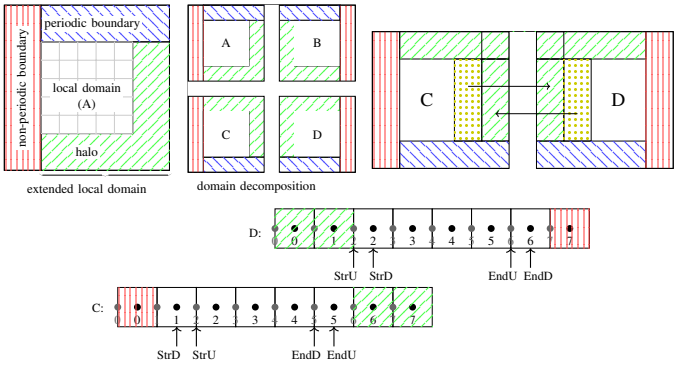


Fig. 1: Top: Local domain with halo and boundary conditions for a single process (left), total domain decomposition (middle), and halo exchanges (right). Bottom: Iteration ranges for two different grid staggerings (StrU to EndU; StrD to EndD).

4) Time Integration: The model state is advanced in discrete time steps, where multi-stepping methods are state-of-the-art for barotropic solvers. Multi-stepping solvers utilize results from several preceding timesteps, and each timestep depends on a fixed set of earlier steps. Codes typically realize this by using an additional index in the arrays of prognostic variables to address the previous steps. This poses additional challenges for correct and efficient code generation, requiring explicit tracking of these time-level dependencies.

C. Communication in Ocean Models

While the barotropic solver operates on two-dimensional fields and therefore requires less computation, it is communication-intensive. Each substep requires halo exchanges for the two barotropic velocity components and the sea-surface height. Since these are typically short messages that are required very frequently, due to the many barotropic steps per timestep, network latency often dominates, making the barotropic solver a performance bottleneck. The total number, communication volume, and code placement of these halo exchanges are the primary targets of the communication optimizations in this work.

III. Parallelization Pipeline and Intermediate Representation

Figure 2 shows our pipeline for automatic distributed parallelization and communication optimization. It operates directly on existing sequential Fortran stencil kernels and consists of three main stages: *lifting*, *optimizing*, and *code generation*.

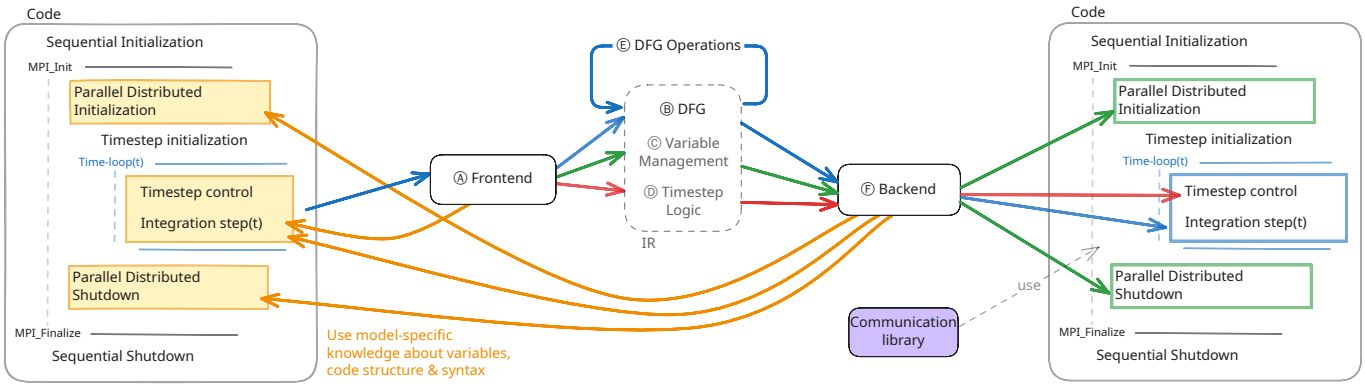


Fig. 2: The integration step's code is uplifted into an IR before applying DFG transformations to parallelize and optimize. The backend converts the result DFG back into code implementing the integration step while adapting initialization and shutdown routines according to the informattion in the Variable Management. The resulting code uses a custom communication library to implement halo exchanges, here using MPI. Letters indicate in which section of this chapter a component will be discussed.

In the first stage, we transform the original Fortran code for a single timestep into an Intermediate Representation (IR), including a Data-Flow Graph (DFG). In the second stage, the DFG is transformed using graph operations that implement distributed parallelization and communication optimization. Finally, the optimized DFG is lowered back to Fortran code, which can be compiled and executed with the surrounding model code using standard compilers. Further information can be found in Poseidon [7], on which our development will be based.

In the following, we describe the individual pipeline stages and the IR, focusing on the components relevant to distributed parallelization and communication optimization.

A. Frontend: Lifting

The lifting phase converts Fortran stencil kernels into our IR. It builds on PSyclone [8], which first generates its own IR (PSyIR). Here, the lifting also provides domain-specific knowledge, e.g., semantic information that cannot be inferred automatically, such as lifetime of data, multi-stepping logic, boundary conditions, grid staggering, and halo configuration. Consequently, it requires *adaptation to each PDE model*, which we do through Python.

The result is a representation that explicitly captures both the global data flow structure and the fine-grained stencil-level data dependencies within kernels.

The IR consists of three main components:

- **DFG** representing dependencies between code blocks,
- **Variable Management (VaM)** storing all variables and metadata, and
- **Time-Stepping Controller** capturing the logic of the outer time loop.

Since each component provides abstraction layers to the model-specificities, a key feature is that both the IR and the optimization process are *agnostic to a particular PDE model*. The following sections provide a more detailed description of these components.

B. Poseidon IR: Variables and Variable Management

The VaM component stores all variables along with metadata such as initialization state, lifetime, halos, boundary conditions, and grid staggering. Variables are classified as:

- **Scalar**: scalar coefficients, indices, and control parameters,
- **GridDataND**: arrays representing structured-grid variables, distributed across processes and with associated halos,
- **MultiStepArray**: collections of grid slices for one variable at different time steps,
- **ArrayND**: multi-dim. arrays not interpreted as grid data.

The IR uses a strict no-aliasing rule: Each memory location is represented by a single variable, which is resolved during uplifting, including cases where different index variables refer to the same time slice of a multi-step array.

C. Poseidon IR: DFG

The data flow within a single generic timestep is represented as a directed data flow hypergraph $H = (V, E)$. Our description will follow the notation used in the book by Sinnen [9].

- V: The set of nodes, each representing a coarse-grained code block analogous to a basic block in traditional compiler IRs.
- E: The set of directed hyperedges representing data dependencies between nodes. An edge $e = (S, T, x)$ connects a set of producer nodes $S \subseteq V$ (tail) to a set of consumer nodes $T \subseteq V$ (head) via variable x. Nodes in T depend on data produced by nodes in S.

For a node $v \in V$, we define incoming and outgoing edges as

$$\mathrm{In}_v = \{e \in E \mid v \in \mathrm{head}(e)\}, \quad \mathrm{Out}_v = \{e \in E \mid v \in \mathrm{tail}(e)\}.$$

*1) **Nodes**:* We distinguish the following types of nodes.

a) Kernel Loop Nodes: They represent stencil computations and are defined by D perfectly nested loops that iterate over the spatial dimensions. Lifting is responsible for bringing loops in the original code into this form. The loop body (the kernel) consists of assignments that update grid variables based on fixed stencil accesses. For communication optimization, we assume kernels are *normalized*. A normalized kernel

applies one stencil operation to compute exactly one output variable and does not contain compound stencils or loop-carried dependencies. Normalization is applied immediately after uplifting to undo manual optimizations and simplify dependency analysis. Although this normalization degrades performance, it also forms the basis for in-depth computational optimizations, as seen in Remy et al. [7]. In the following, we only describe DFG nodes relevant for communication and refer to Remy et al. [7] for more information on omitted information on "Computation" and "Black Box Nodes".

b) Boundary Condition Nodes: They represent dynamic boundary updates, such as Neumann conditions, that are applied repeatedly during time integration, identified in our case by $(D-1)$ dimensional loops.

c) Communication Nodes: They represent halo exchanges for a single grid variable and contain abstract communication requirements, such as halo size and number of layers. Later transformations split these nodes into start and end phases, forming the basis for asynchronous overlap.

d) Source and Sink Nodes: Source and sink nodes mark the beginning and end of a timestep in the DFG. The source node introduces all data inflow required for the timestep. The sink node collects all data outflow that persists beyond the timestep. The different DFG nodes are connected by hyperedges representing their data-flow dependencies.

*2) **Edges**:* An edge from node N to node M labeled with array variable x indicates a respective dependency on x with data created by N and consumed by M. For arrays, a dependency between two array variables exists if any of their elements have dependencies. The exact spatial dependency relationships between grid elements are described using stencils. We distinguish two types of edges[1]:

a) Variable Edges: Variable edges represent true read-after-write dependencies with one tail node writing to an array variable and all the edges' head nodes reading it. They are used to infer when halo exchanges are required, as communication is needed only if an array variable written by one process is subsequently read by another.

b) False Dependence Edges: False dependence edges represent write-after-write and write-after-read dependencies. They do not induce communication, but are necessary to ensure the correct execution order during code generation.

D. Poseidon IR: Time Stepping

The DFG represents the data flow within a single loop iteration using generic multi-step indices and is generic to all timesteps. The timestepping controller contains a list of time indices, along with their corresponding mappings to logical timesteps. It also tracks additional time-related array variables such as the loop index t_n.

[1]For the sake of better readability, we skip in this section the repetitive exclusion of scalar variables that are naturally without stencil access dependencies

E. Optimization with DFG Transformations

After lifting and normalization, we encode all information required for optimization in the DFG. Note, this representation is agnostic to the used PDE solver. The optimization itself will be presented in a dedicated Sec. V-B, providing sufficient details on the set of graph transformations that derive correct and efficient communication patterns from this representation.

F. Backend: Code Generation

After optimization, the DFG is lowered back to Fortran code through Poseidon [7]. Shared-memory parallelization is implemented using backends for parallel programming models, such as OpenMP, see Remy et al. [7] for details.

We point out that each model requires its own backend, as, for example, a one-time initialization for optimized communication necessitates the injection of code at specific initialization routines, which are again specific to each model. The particular implementation of the halo exchanges uses a custom communication library described in Sec. VI-A.

IV. Requirements for Correct Distributed Execution

As outlined in Sec. II, each process updates its local domain, but often requires neighbor data for stencil operations, which is exchanged through halos. In practice, this means that specific regions of each array must be locally available at the correct program point. In this section, we define when communication is required and how correctness can be ensured in distributed stencil computations.

A. Definitions

We use a simple communication model to describe which parts of each array must be locally available and which subarrays of an array are produced by each operation.

a) Representing Subarrays: First, we define the ranges for each subarray of a local domain for node v by

$$L(v) = \left(l_i^{\text{start}}, l_i^{\text{end}}\right)_{i=1}^{D}\Big|_v .$$

Then, we represent subarrays of an D-dimensional grid as axis-aligned boxes relative to a local domain of a process:

$$R(v) = \left(o_i^{\text{start}}, o_i^{\text{end}}\right)_{i=1}^{D}\Big|_v ,$$

where o_i^{start} and o_i^{end} denote offsets from the local domain bounds along the i-th dimension. The corresponding set of grid points for dimension D is:

$$\{x \mid l_i^{\text{start}} - o_i^{\text{start}} \leq x_i \leq l_i^{\text{end}} + o_i^{\text{end}},\ \forall \text{ dimensions } i\}.$$

Kernel loop bounds are represented similarly. We define bounds for DFG node v by

$$B(v) = \left(a_i, b_i\right)_{i=1}^{D}\Big|_v ,$$

where the loop iterates from $l_i^{\text{start}} - a_i$ to $l_i^{\text{end}} + b_i$ in each dimension. To give an example for dimension i, a value of zero means the iteration range matches the local domain exactly. Positive offsets extend the range into the halo: The tuple $(1, 1)$,

for example, adds one halo layer on each side, while $(2, 2)$ adds two layers for the respective dimension. Negative offsets indicate a smaller range than the local domain (e.g., $(-1, -1)$ excludes one layer from both sides).

b) Input Footprint: The input footprint $F_{\text{in}}(v, x)$ of array x at node v is the region of x that must be locally available to execute the stencil of v over its iteration range $R(v)$. It is computed by expanding $R(v)$ by the maximum of all stencil offsets applied to x of node v:

$$F_{\text{in}}(v, x) = \left(\max_{s \in \mathcal{S}_x}(o_i^{\text{start}} - s_i),\ \max_{s \in \mathcal{S}_x}(o_i^{\text{end}} + s_i) \right)_{i=1}^{D} \Bigg|_{v,x},$$

where s_i are the stencil offsets in dimension i, and $\mathcal{S}_x$ is the set of all stencils the kernel applies to x.

c) Output Footprint: The output footprint $F_{\text{out}}(v)$ is the region of its output array x computed by node v. It equals the iteration range of the node, therefore given by

$$F_{\text{out}}(v) = F_{\text{out}}(v, x) = R(v).$$

B. Communication Model in a Distributed Setting

After partitioning, we start with the most naive approach, where each process computes values only for its (non-overlapping) local domain, while its halos contain values from neighboring processes to satisfy stencil dependencies. At the end of each timestep, only the values of prognostic arrays that persist across timesteps must be computed. Each process, therefore, must ensure the computation of the array values of its entire local domain. Formally, since all and only prognostic arrays are connected to the sink node, we obtain $F_{\text{in}}(\text{sink}, x) = 0$.

C. Correctness Conditions

Based on the previous formalization, we can specify conditions a DFG with footprints and iteration ranges needs to fulfill for encoding correct distributed parallelization, where replacing the inequalities with equality would even exclude computation of data not needed.

1) **All necessary results for prognostic arrays:**

$$F_{\text{in}}(\text{sink}, x) \geq 0, \quad \text{for every prognostic } x.$$

2) **Consistency of iteration ranges:** Each node v must produce enough data for all of its successors c:

$$F_{\text{out}}(v, x) \geq \max_{c \in \text{children}(v)} F_{\text{in}}(c, x).$$

3) **Bounded Footprints:** All footprints must remain within the extended range allowed for the array; in particular, we cannot require more halo layers than allocated:

$$F_{\text{in}}(v, x) \leq \text{maxHalos}(x) \ \wedge \ F_{\text{out}}(v, x) \leq \text{maxHalos}(x).$$

4) **Time-stepping Consistency:** The first node in a timestep must only require data available from initialization F_{init} or the previous timestep:

$$F_{\text{out}}(\text{source}, x) \leq F_{\text{in}}(\text{sink}, x)$$

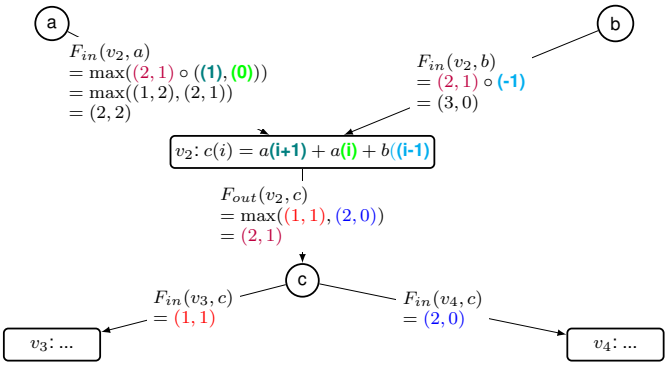


Fig. 3: Example of a 1D footprint propagation with 1D stencils (stencil-related parts in boldface): input requirements at nodes v_3 and v_4 are propagated backward (bottom-to-top) through the graph. $\circ$ describes applying a stencil to a footprint.

D. Footprint Propagation in the DFG

We can compute the required footprints for all nodes by propagating footprint requirements backward through the DFG, starting from the sink node, using Condition (1) and applying Condition (2) transitively. This ensures that each node produces all data needed by its successors. Conditions (3) and (4) are then used to verify that the resulting footprints remain within the allowed bounds and respect the time-stepping constraints. Additionally, propagation uses the following rules for node v:

- **Kernel loop nodes:** Following the definition of the input footprint, we apply all corresponding stencils s^x to the iteration range (which equals the output footprint) of a kernel loop.

$$F_{\text{in}}(v, x) = \left(\left(\max_{s \in \mathcal{S}_x}(F_{\text{out}}^{\text{start}}(v)_i - s_i),\ \max_{s \in \mathcal{S}_x}(F_{\text{out}}^{\text{end}}(v)_i + s_i) \right) \right)_{i=1}^{n}$$

- **Computation and black-box nodes:** These nodes do not modify distributed grid arrays. Footprints are propagated unchanged:

$$F_{\text{in}}(v, x) = F_{\text{out}}(v)$$

- **Communication nodes and boundary conditions:** Both need only their entire local domain to set halo or boundary values:

$$F_{\text{in}}(v, x) = 0.$$

The resulting footprints define iteration ranges required for correctness and form the basis for injecting and optimizing halo exchanges detailed in the next Section.

V. Communication Inference and Optimization

Based on the previous section, we are now ready to infer and optimize communication. Before describing the individual transformations, we first outline the general idea behind these optimizations.

A. General Idea of Communication Optimization

Before starting with a detailed description of the realization of the optimization process, we briefly describe the targets of this process to which we will refer to later.

T1 Communication Avoidance: By analyzing data dependencies, we can determine when communication is truly necessary and apply strategies to reduce its frequency:

T1.a Extending Halo Sizes: Instead of performing frequent exchanges of small halos, halos can be extended by adding additional layers. With these wider halos, the iteration ranges are extended to allow stencil operations to be applied to the halo regions, thereby updating their values. This introduces redundant computation, because some halo values are computed both locally and on their owning process. However, this significantly reduces the frequency of halo exchanges by enabling multiple stencil applications between communications.

T1.b Eliminating Redundant Halo Exchanges: Halo exchanges that operate on the same array without intermediate updates can be merged. This optimization applies both within a single time step and across time steps.

T2 Latency Hiding: The latency from the remaining exchanges can be hidden by using non-blocking and asynchronous communication and overlapping the communication with computation. In this way, the CPU performs computations, which are independent of the communicated data, while communication is executed by the network in the background.

B. Communication-related DFG Operations

Next, we describe, step-by-step, the necessary DFG operations that will be applied successively to obtain our optimized communication scheme.

C. Operation 1: Injecting Exchanges

The goal of this step is to inject a halo exchange before each kernel accesses a halo of a local domain, creating an initially inefficient but valid communication scheme. This is done in two steps:

a) Set Iteration Ranges to Local Domains: Initially, each kernel loop iterates solely over the local domain of its output, making its footprint equal to the output local domain of the array:

$$R(v) = F_{\text{out}}(v) = 0.$$

b) Insert Required Exchanges: Before a kernel loop accesses data outside its local domain, the corresponding halo region must be updated, with the required halo size being equal to the input footprint of the array. Thus, before each node v, we insert a communication node (initially combining send & recv) for each input array x with halo size given by $F_{\text{in}}(v, x)$. This guarantees that all required halo layers are available for the computation.

This operation inserts halo exchanges that are required for correctness as well as empty exchanges (when $F_{\text{in}}(v, x) = 0$)). The empty communication nodes serve as placeholders, allowing later optimizations that indeed require communication.

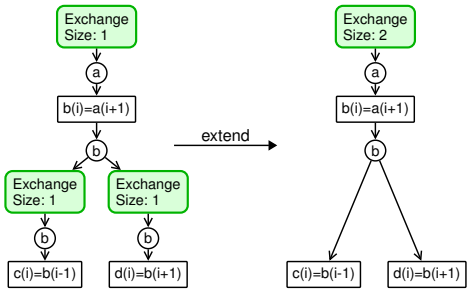


Fig. 4: Example of applying the halo extending operation. Both exchanges at the bottom of the DFG on the left can be removed by extending the halo size of b, their parent node's iteration range (not shown), and the size of the halo exchange at the top. This results in the DFG on the right.

D. Operation 2: Replacing Exchanges by Extending Halo Layers

After the initial injection phase, we begin with the first optimization step, which realizes T1a by reducing the number of halo exchanges through fewer, larger transfers moved as early as possible within the timestep. This is achieved by extending halos, allowing multiple small exchanges to be replaced by fewer, larger ones.

1) Extending Halo Layers: The basic idea is to remove communication nodes from the DFG and compensate by extending the iteration ranges and halo sizes of their ancestor nodes. This ensures that required subarrays remain available, now through redundant computation rather than additional exchanges.

We take the input DFG and remove communication nodes one by one, extending iteration ranges and halos of ancestor nodes to cover the required regions, as in the example in Figure 4. To infer what changes are required after removing a communication node, we use the propagation approach from Section IV-A.

To remove a given communication node C_i:

1) Simulate removing C_i by recalculating footprints and iteration ranges of parent nodes.
2) Propagate changes upward as long as a parent node's F_{in} increases.
3) If another communication node is reached, resize its halo to the required F_{out} and stop propagation.
4) If any correctness condition (Sec. IV-A) is violated at any point, continue with the next node (since C_i cannot be removed).
5) If successful, the node can be removed safely, apply updates, and delete C_i from the DFG.

Applying this procedure to all communication nodes in reverse topological order moves communication to the beginning of a timestep, reducing the number of exchanges while preserving correctness.

2) Staggered Grids and Boundary Conditions: As described in V-D2, the use of staggered grids means that different arrays can be defined on different iteration ranges. To handle

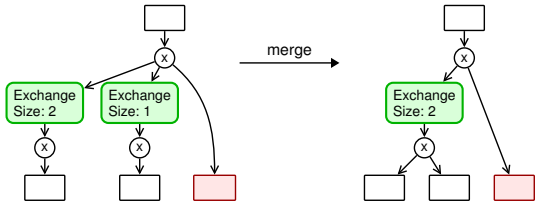


Fig. 5: The two exchanges operating on the same data (left) are merged into one with the maximum of their size (right).

this, each initial footprint is represented as a set of bounding boxes, one per grid staggering.

At the edges of the grid, boundary conditions apply. Periodic boundary conditions can be easily implemented as halo exchanges with the "neighbor" located on the opposite edge of the grid. In the case of non-periodic static boundary conditions (e.g., Dirichlet boundary conditions set during initialization) on an edge of the grid, additional care is required. To avoid overwriting such values, we take the extended local domain as shown in Figure 1 as the maximum extent of the iteration ranges for arrays with non-periodic boundary conditions.

E. Operation 3: Merging Redundant Exchanges

Since there is no check for whether a value has already been updated, redundant communication can occur when several nodes require the same exchange. To avoid this, some communication nodes can be eliminated by merging them with their identical "twin" exchanges, as illustrated in Fig. 5, targeting T1.b.

We identify redundant exchanges by checking, for each communication node, if there is a sibling node exchanging the same array. In this case, as both operate on identical data (their parent nodes'output), they are merged into a single node by using the maximum of the ranges of the nodes.

F. Operation 4: Inter-Iteration Merging

So far, communication transformations have been applied only *within* a single timestep t_n. Analyzing data dependencies across timesteps allows us to identify and eliminate additional redundant exchanges implementing T1.b.

1) Merging Redundant Exchanges Across Timesteps: Two exchanges in different timesteps can be merged if both of the following conditions hold:

- Same array: Both exchanges refer to the same array. For multistep arrays, this requires resolving time-level indices.
- No intermediate writes: The array is not modified between the two exchanges.

In this case, the latter exchange is redundant and can be removed by redirecting its outgoing edges to the earlier exchange and adjusting its halo size, (see the example in Figure 6).

2) Hoisting Loop-Invariant Exchanges: If an array is never written inside the timestep loop, all its exchanges can be collapsed into a single exchange. This exchange can be hoisted out of the loop into the initialization phase or eliminated entirely if prior halo initialization can be assumed.

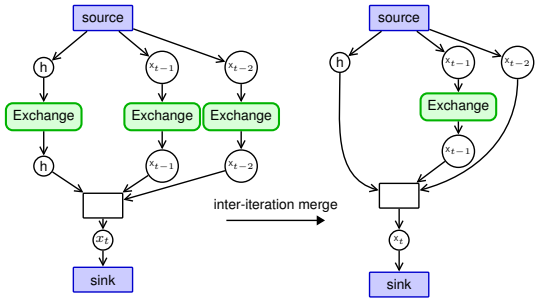


Fig. 6: An inter-iteration merge removing exchanges for a constant (leftmost exchange on the left) and an older time-slice of a prognostic array (rightmost exchange on the left).

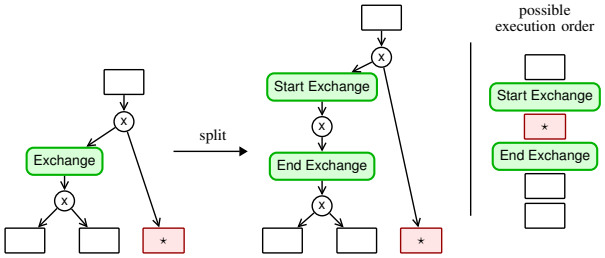


Fig. 7: Splitting a halo exchange into start and end nodes (left) enables scheduling an independent kernel loop marked by $\star$ between them, allowing overlap with communication (right).

G. Operation 5: Splitting Halo Exchanges for overlapping

We realize T2, overlapping communication and computation, by transforming each synchronous halo exchange into two asynchronous operations: a *start* and an *end* communication node. The start node initiates the halo exchange, while the end node waits for completion. Figure 7 shows an example of this transformation.

As a result, any node that is not dependent on the exchanged data (i.e., not a descendant of the end node) can be executed between the start and end operations. This creates an opportunity to execute independent computations while communication progresses in the background, reducing idle time and improving overall performance.

H. Operation 6: Preposting the Starts of Exchanges

The potential overlap window (for target T2) can be extended further by moving the start of a halo exchange to the prior timestep if the dependencies allow it (see the example in Figure 8). A start node for array x_n can be shifted to timestep $n-1$ by inserting a new start node after the last write to x_{n-1}.

I. Complete Optimization Algorithm

The transformations described above are combined into the following optimization workflow:

1) **Inject Exchanges:** Inject all halo exchanges communication nodes.
2) **Extend Halos:** Traverse the DFG in reverse topological order. For each communication node, attempt to remove it by extending the halos of ancestor nodes.

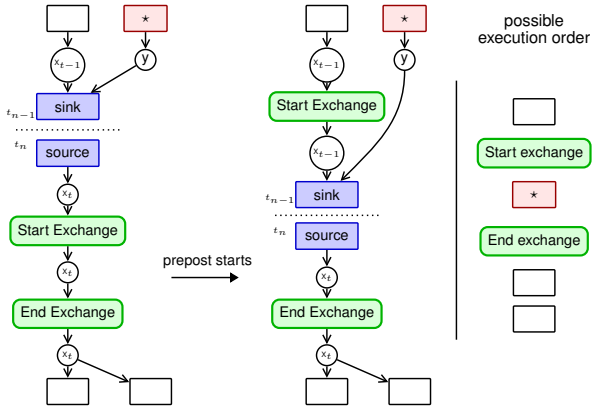


Fig. 8: Preposting moves the start of the halo exchange from the iteration start (directly after the source node) over the inter-iteration boundary (dotted line) to the end of a prior iteration, inserting it directly before the sink. This creates an opportunity to execute an independent kernel loop (marked $\star$) between the start and end phases, enabling potential overlap (right image).

3) **Merge Duplicates:** Merge pairs of twin nodes to reduce redundant communication.
4) **Inter-Iteration Merging** Merge redundant exchanges across timesteps and hoist loop-invariant exchanges outside the timestep loop.
5) **Split to Overlap:** Split all halo exchanges into asynchronous start and end nodes to hide latency.
6) **Prepost Starts:** Move start nodes across timestep boundaries to further extend the communication-computation overlap window.

VI. Backend and Code Generation

The backend generates code for declaring, initializing, and deleting required array variables, executing the timestep, and implementing communication. We assume that either the original application already performs domain decomposition (Section II-B3) and handles MPI initialization and finalization (as used in the present work), or that this functionality is provided by a model-specific implementation.

A. Communication Library

We decouple the placement of communication operations in the DFG from their concrete implementation. In the backend, we will utilize a communication library written in modern Fortran. This library can be used independently of the present work to implement halo exchanges for ocean models on a structured grid, utilizing stencil codes. It conceals the details of how communication is actually performed from the model developers (e.g., non-blocking, persistent communication, derived datatypes, with/without manual buffers), requiring them to specify the subarrays and ranks of neighborhoods solely. It consists of three phases: 1. initialization (providing information on arrays and subarray ranges to be exchanged, called only once for each stencil pattern), 2. begin data exchange, and 3. waiting until the exchange is complete. This enables us to implement all kinds of optimizations for distributed memory communication while hiding from users the details of how the communication is actually done.

B. Ordering

During code generation, the DFG is converted back into Fortran code using topological sorting [10] of the nodes of the DFG, taking into account both true and false dependency edges. The DFGs considered in this work allow different topological orderings, and identifying the "best" one is non-trivial. For this work, we use a simple greedy scheduling strategy: Starting from the source, we repeatedly select any node whose predecessors have already been selected. Start communication nodes are given the highest priority, end communication nodes are deferred until no other nodes are available, and all remaining nodes follow the ordering of the original code when possible.

VII. Evaluation

We evaluate the correctness and performance of the automatically generated parallel code on a barotropic solver implementing the numerics from CROCO's 2D soliton benchmark, with periodic boundary conditions enabled.

A. Experimental Setup

We conduct our experiments on the SuperMUC-NG Phase 1 cluster (Intel Xeon Platinum 8174, 24 cores per socket, two sockets per node, Intel OmniPath 100 Gb/s interconnect). The code is compiled with Intel IFX (version 2025.1.0, options: `-O3 -xHost -ipo`) and Intel MPI (version 2021.15).

Based on preliminary benchmarks of the used system, we employ MPI non-blocking communication with manual buffers (storing non-consecutive data compactly before communication) in the communication library. Also based on benchmarks, we disable the progress thread to perform better once reaching the strong scaling limits, where eager mode performs better.

As described in Sec. III, we lift the sequential code, apply normalization and distributed-memory parallelization and communication optimization, insert OpenMP shared-memory parallelization, and generate the final hybrid MPI+OpenMP implementation.

We use one MPI process per node with $\min(48, \frac{\text{cells per node}}{2{,}048})$ OpenMP threads, thereby counteracting threading overheads due to low workloads. We parallelize kernel loops using OpenMP and initialize arrays initialized using a first-touch policy. We perform multiple warmup timesteps before benchmarking to avoid cold-start effects and MPI lazy initialization overheads.

We compare several variants, incrementally applying communication optimizations, verifying correctness against the sequential reference, and measuring performance for two grid resolutions: 1024^2 and 4096^2 over 8192 timesteps. To reduce computational cost, the two least optimized variants are only executed for $1,024$ timesteps.

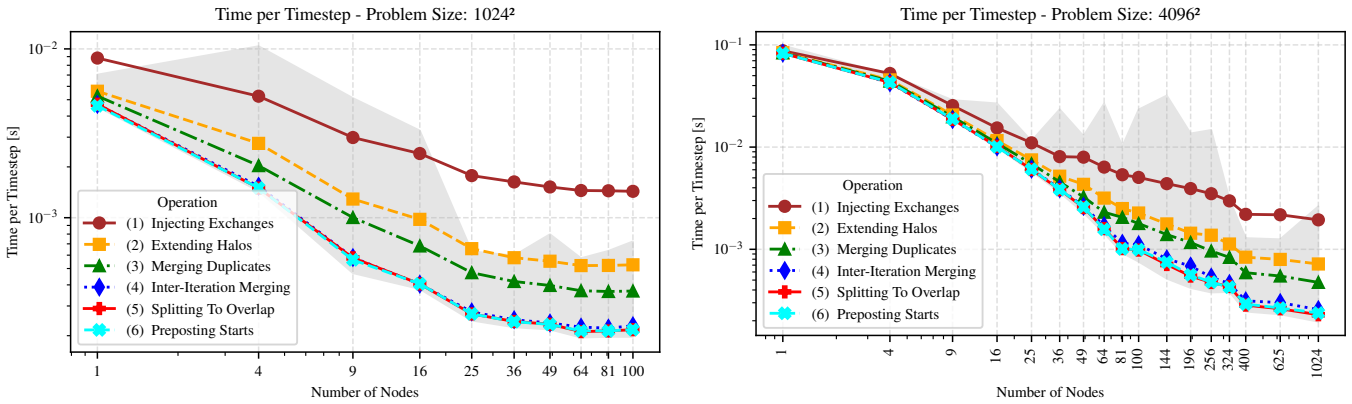


Fig. 9: Time per timestep for different optimization stages and two problem sizes. The shaded area shows the min/max range for the best variant at each node count. We observe a high variance for the maximum communication time.

B. Generated Code

A relation between the successive optimizations and the number of halo exchanges and opportunities for overlap is given in Tbl. I. Initially, simple parallelization by injecting halo exchanges for every halo access produces 68 exchanges per timestep. This number is reduced significantly by extending halo sizes and merging identical exchanges, first within a timestep and then across timesteps. Splitting enables asynchronous communication, creating the opportunity to overlap communication and computation. After all transformations, only three exchanges per timestep remain, transferring one or two halo layers depending on the variable, matching the optimal hand-tuned code for one step of this solver.

TABLE I: Number of halo exchanges per timestep, average number of halo layers per exchange, and number of Kernel Loops (KL) between start and end of each exchange (only displayed for last two optimization steps).

Version	Exchanges	Avg. Layers	Overlap in KL
Injecting Exchanges	68	1.00	-
+ Extending Halos	19	1.21	-
+ Merging Duplicates	10	1.40	-
+ Inter-Iteration Merging	3	1.33	-
+ Overlapping Exchanges	3	1.33	(0, 1, 2)
+ Preposting Starts	3	1.33	(1, 1, 20)

C. Performance Results

1) Iteration Time: We first investigate performance benchmarks, as shown in Figure 9, which displays run times for different optimization stages and two problem sizes. For all variants, run times decrease quickly with increasing node counts as both the problem size per node and the message sizes decrease. For both problem sizes beyond approximately 200^2 cells per node (about 800 cells per core), improvements taper off because both computation and communication cease to improve as short messages are dominated by network latency, and computation cannot fully utilize all available cores and cache.

Compared to the naive first variant *(1)*, communication-avoidance optimizations *(2, 3, 4)* result in substantial run time reductions due to their significantly reduced number of exchanges. Latency hiding is also successful with the asynchronous variant *(5)*, after splitting, and it performs better compared to the synchronous variant before splitting. This difference, however, is not as pronounced. Preposting starts *(6)* does not consistently decrease run times.

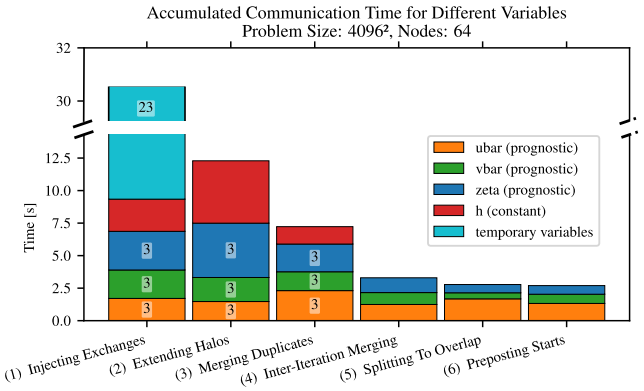


Fig. 10: Breakdown of communication times by variable. Numbers indicate how many variables are included.

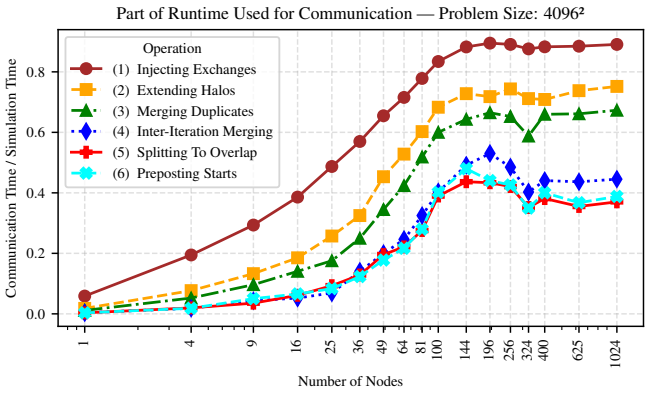


Fig. 11: Fraction of run time spent in communication.

2) Composition of communication time: Figure 10 shows a decomposition of the total communication time for a given case into different variables, showing the causes of decreases in communication time after applying optimizations. At the start *(1)*, there are exchanges for the last three timestep results of the prognostic variables, as well as 23 temporary variables and one constant. Through halo extension *(2)*, we first remove exchanges of all temporary variables, decreasing the communication time by 60%. The next two optimizations reduce the number of exchanges for certain variables *(3)* and then remove exchanges for both constants and prognostic variables that were exchanged in past timesteps *(4)*. Splitting halo exchanges into an initiating and a finalizing phase *(5)* enables asynchronous communication, resulting in less communication time for two of the three exchanges. However, overlap for the remaining exchange is limited because the available computation between its start and end phases is very short, containing no kernel (see Table I). Preposting the start of the exchange *(6)* to increase the size of the potential overlap window is unable to improve performance because the computation that can potentially be overlapped with is still comparatively small relative to the communication time. Additionally, this repositioning reduces overlap between the different exchanges, increasing communication times for other variables. For this reason, the last optimization operation (6) fails to provide any consistent advantages for our code and actually decreases the performance in most cases.

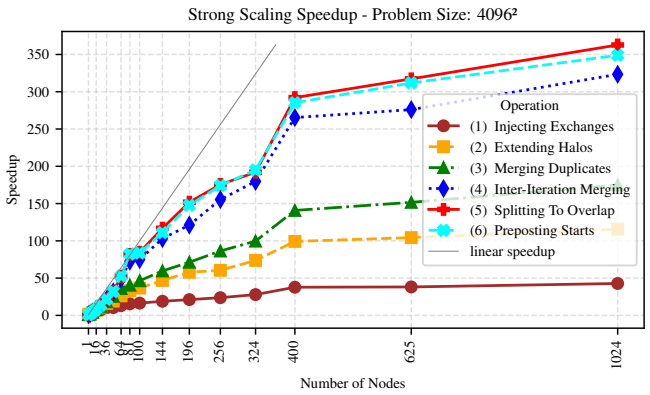


Fig. 12: Strong-scaling speedup for all optimization variants.

3) Scalability: Next, we investigate the parallel speedup relative to the single-node, non-distributed original version for the larger problem size with results displayed in Fig. 12. The best variant reaches a speedup of over 350 on $1,024$ nodes, compared to less than 50 for the naive version. Communication avoidance provides the largest benefit, but overlapping still makes a noticeable difference at large node counts, where communication times are significant.

Finally, we investigate the fraction of total run time spent in communication with results displayed in Fig. 11. For the naive variant, communication dominates at large node counts, accounting for roughly 90% of the total run time. After applying our optimizations, this drops to below 40%. Note that communication time appears even for a single node because periodic boundary conditions (which involve halo exchanges if multiple nodes are involved) are included in these measurements, even though they do not involve MPI on a single node. Interestingly, the fraction decreases noticeably between 196 and 324 nodes. This is due to relatively small improvements in computation time compared to larger reductions in communication time for these problem sizes. We account for this by the eager send threshold of 4096 B, corresponding to message sizes generated by two-layer halo exchanges at problem sizes near 256^2, where communication becomes more efficient.

VIII. Related Work

Automatic optimization of distributed stencil computations has been only partly investigated with most existing approaches requiring rewriting applications into domain-specific abstractions or adding annotations to expose communication.

Several stencil DSLs can generate distributed-memory communication once computations are expressed in their abstractions. OPS and OP2 [11], [12] perform communication optimizations similar to ours, but do not perform static analyis, instead depending on a runtime system and require kernels to be rewritten against their APIs. Physis [13], Devito [14], YASK [15], Distributed Halide [16] all generate distributed code from schedule annotations. All approaches require substantial refactoring into DSL APIs and often rely on specialized run times.

PGAS languages such as Coarray Fortran, UPC, Chapel, and XcalableMP provide global-view programming with one-sided communication. Coarray Fortran [17] and XcalableMP [18] are syntactically close to Fortran but still require code restructuring and also largely manual halo communication. Chapel can infer and optimize halo exchanges [19], [20], but does not provide time-loop–aware communication optimization and requires rewriting.

Polyhedral and IR-based systems, including Pluto-Distmem [21], Tiramisu [22], and Arachne [23], can generate distributed communication from structured representations, but require explicit affine loops, scheduling directives, or annotations.

In contrast, our approach lifts existing Fortran stencil kernels, reconstructs domain- and time-level dependencies without DSL rewriting or annotations, and generates optimized distributed-memory communication directly from the original code.

IX. Conclusions

We introduced an approach for automatic distributed-memory parallelization and communication optimization for a single timestep of structured stencil codes directly from existing Fortran code. For the first time, this allows to lift up an existing solver code in Fortran to a data-flow IR to infer communication requirements formally, and apply graph-based transformations optimizing for minimizing exchanges and enabling overlapping communication and computation.

This is especially important for ocean modelling, as production ocean models support a multitude of different configurations that are implemented through extensive preprocessor usage. Since these different configurations have distinct data dependencies and communication requirements, manual optimization for all configurations is infeasible. In contrast, our approach provides the first important step towards a fully automated and optimized solution.

We evaluated our method on a 2D barotropic solver, which poses a particular challenge due to its low workload, revealing substantial performance gains for strong scaling with over $350\times$ speedup on $1,024$ nodes by using our fully automatic distributed memory parallelization.

X. Acknowledgements

The authors gratefully acknowledge the Gauss Centre for Supercomputing e.V. (www.gauss-centre.eu) for funding this project by providing computing time on the GCS Supercomputer SuperMUC-NG at Leibniz Supercomputing Centre (www.lrz.de).

We like to thank funding provided through the Poseidon IRGA funding and the Inria Crocodiles associate team.

References

[1] *Frontier: Exploring Exascale*. DOI: 10.1145/3581784.3607089.

[2] J. S. Vetter, R. Brightwell, M. Gokhale, *et al.*, "Extreme Heterogeneity 2018 - Productive Computational Science in the Era of Extreme Heterogeneity: Report for DOE ASCR Workshop on Extreme Heterogeneity," USDOE Office of Science (SC), Washington, D.C. (United States), Tech. Rep., Dec. 2018. DOI: 10.2172/1473756.

[3] F. Auclair, R. Benshila, L. Bordois, *et al.*, *Coastal and Regional Ocean COmmunity model*, Zenodo, Apr. 2024. DOI: 10.5281/zenodo.11036115.

[4] G. Madec, M. Bell, R. Benshila, *et al.*, "NEMO Ocean Engine Reference Manual," Dec. 2024. [Online]. Available: https://zenodo.org/records/14515373.

[5] A. F. Shchepetkin and J. C. McWilliams, "Computational Kernel Algorithms for Fine-Scale, Multiprocess, Longtime Oceanic Simulations," in *Handbook of Numerical Analysis*, ser. Special Volume: Computational Methods for the Atmosphere and the Oceans, R. M. Temam and J. J. Tribbia, Eds., vol. 14, Elsevier, Jan. 2009, pp. 121–183. DOI: 10.1016/S1570-8659(08)01202-0.

[6] A. Arakawa and V. R. Lamb, "A Potential Enstrophy and Energy Conserving Scheme for the Shallow Water Equations," *Monthly Weather Review*, vol. 109, no. 1, pp. 18–36, Jan. 1981, ISSN: 1520-0493, 0027-0644. DOI: 10.1175/1520-0493(1981)109⟨0018:APEAEC⟩2.0.CO;2.

[7] J. Remy, H. Brunie, M. Schreiber, *et al.*, "Numerics-driven uplifting, Automatic Parallelization, and Performance Optimizations with Deep Kernel Fusion for Ocean Models on Heterogeneous Architectures," 2025. [Online]. Available: https://hal.science/hal-05061001.

[8] S. Siso, A. R. Porter, and R. W. Ford, "Transforming Fortran weather and climate applications to OpenCL using PSyclone," in *Proceedings of the 2023 International Workshop on OpenCL*, ser. IWOCL '23, New York, NY, USA: Association for Computing Machinery, Apr. 2023, pp. 1–8, ISBN: 979-8-4007-0745-2. DOI: 10.1145/3585341.3585360.

[9] O. Sinnen, *Task Scheduling for Parallel Systems*, 1st ed. John Wiley & Sons, Ltd, 2007. DOI: 10.1002/0470121173.

[10] A. B. Kahn, "Topological sorting of large networks," *Commun. ACM*, vol. 5, no. 11, pp. 558–562, Nov. 1962, ISSN: 0001-0782. DOI: 10.1145/368996.369025.

[11] I. Z. Reguly, G. R. Mudalige, M. B. Giles, D. Curran, and S. McIntosh-Smith, "The OPS Domain Specific Abstraction for Multi-block Structured Grid Computations," in *2014 Fourth International Workshop on Domain-Specific Languages and High-Level Frameworks for High Performance Computing*, Nov. 2014, pp. 58–67. DOI: 10.1109/WOLFHPC.2014.7.

[12] I. Z. Reguly, G. R. Mudalige, and M. B. Giles, "Loop Tiling in Large-Scale Stencil Codes at Run-Time with OPS," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 4, pp. 873–886, Apr. 2018, ISSN: 1558-2183. DOI: 10.1109/TPDS.2017.2778161.

[13] N. Maruyama, T. Nomura, K. Sato, and S. Matsuoka, "Physis: An implicitly parallel programming model for stencil computations on large-scale GPU-accelerated supercomputers," in *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '11, New York, NY, USA: Association for Computing Machinery, Nov. 2011, pp. 1–12, ISBN: 978-1-4503-0771-0. DOI: 10.1145/2063384.2063398.

[14] F. Luporini, M. Louboutin, M. Lange, *et al.*, "Architecture and Performance of Devito, a System for Automated Stencil Computation," *ACM Transactions on Mathematical Software*, vol. 46, no. 1, pp. 1–28, Mar. 2020, ISSN: 0098-3500, 1557-7295. DOI: 10.1145/3374916.

[15] C. Yount, J. Tobin, A. Breuer, and A. Duran, "YASK-yet another stencil kernel: A framework for HPC stencil code-generation and tuning," in *Proceedings of the Sixth International Workshop on Domain-Specific Languages and High-Level Frameworks for HPC*, ser. WOLFHPC '16, Salt Lake City, Utah: IEEE Press, Nov. 2016, pp. 30–39, ISBN: 978-1-5090-6156-3.

[16] T. Denniston, S. Kamil, and S. Amarasinghe, "Distributed Halide," in *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '16, New York, NY, USA: Association for Computing Machinery, Feb. 2016, pp. 1–12, ISBN: 978-1-4503-4092-2. DOI: 10.1145/2851141.2851157.

[17] J. Reid, "Coarrays in the next fortran standard," in *ACM SIGPLAN Fortran Forum*, ACM New York, NY, USA, vol. 29, 2010, pp. 10–27.

[18] H. Murai and M. Sato, "An Efficient Implementation of Stencil Communication for the XcalableMP PGAS Parallel Programming Language," in *7th International Conference on PGAS Programming Models*, 2013, p. 142.

[19] E. Kayraklioglu, E. Ronaghan, M. P. Ferguson, and B. L. Chamberlain, "Locality-Based Optimizations in the Chapel Compiler," in *Languages and Compilers for Parallel Computing*, X. Li and S. Chandrasekaran, Eds., vol. 13181, Cham: Springer International Publishing, 2022, pp. 3–17, ISBN: 978-3-030-99371-9 978-3-030-99372-6. DOI: 10.1007/978-3-030-99372-6_1.

[20] M. P. Ferguson and D. Buettner, "Caching Puts and Gets in a PGAS Language Runtime," in *2015 9th International Conference on Partitioned Global Address Space Programming Models*, Washington, DC, USA: IEEE, Sep. 2015, pp. 13–24, ISBN: 978-1-5090-0185-9. DOI: 10.1109/PGAS.2015.10.

[21] R. Dathathri, C. Reddy, T. Ramashekar, and U. Bondhugula, "Generating efficient data movement code for

heterogeneous architectures with distributed-memory," in *Proceedings of the 22nd International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT '13, Edinburgh, Scotland, UK: IEEE Press, Oct. 2013, pp. 375–386, ISBN: 978-1-4799-1021-2.

[22] R. Baghdadi, J. Ray, M. B. Romdhane, *et al.*, "Tiramisu: A Polyhedral Compiler for Expressing Fast and Portable Code," *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 193–205, Feb. 2019. DOI: 10.1109/CGO.2019.8661197.

[23] Y. He, H. Zhao, W. Cui, S. Zhang, Q. Chen, and M. Guo, "ARACHNE: Optimizing Distributed Parallel Applications with Reduced Inter-Process Communication," *ACM Trans. Archit. Code Optim.*, vol. 22, no. 2, 52:1–52:26, Jun. 2025, ISSN: 1544-3566. DOI: 10.1145/3716871.